

Relaytic-AML: A Local-First, Agent-Assisted Evaluation Lab for Financial-Crime Machine Learning

Tobias Gehra
Independent Researcher
GitHub: `ML-Enthusiast-de`
`t.gehra.ai@gmail.com`

July 2026

Abstract

Anti-money laundering (AML) machine-learning experiments are difficult to audit when data residency, temporal validity, graph provenance, agent assistance, review capacity, and public reporting are handled separately. Relaytic-AML is a local-first evaluation lab in which capability-scoped agents and deterministic harnesses turn local runs into provenance-bearing measurements and bounded release decisions. Temporal PaySim and Elliptic workflows, an Elliptic2 reference workflow, and deterministic governance fixtures evaluate the architecture. The selected PaySim and Elliptic test PR-AUC point estimates are 0.6388 and 0.6688. The Elliptic2 context estimate is 0.9432 ± 0.0009 , alongside a published RevClassifyDS reference of 0.9740. The tasks are not a shared leaderboard. The contribution is the evaluation, governance, and reproducibility architecture rather than a new detector or detector-superiority result.

1 Introduction

AML modeling is a rare-event, high-stakes evaluation problem. The input may be a stream of mobile-money transfers, card events, account activity, wire messages, customer profiles, or blockchain transactions. Suspicion often appears as a temporal or network pattern: rapid movement of newly received funds, repeated transfers through related accounts, structured amounts, new counterparties receiving many payments, activity inconsistent with a customer profile, or cryptocurrency flows involving higher-risk services and jurisdictions. FATF and FFIEC material describes such pattern-oriented red flags in operational terms [Financial Action Task Force, 2020, Federal Financial Institutions Examination Council, 2014].

Because AML suspicion is often pattern-based, an isolated model metric is easy to misread. A precision-recall area under the curve (PR-AUC) estimate or a precision-at-review-budget estimate becomes interpretable only when the evaluation record shows which data stayed local, which fields were available at decision time, how temporal or graph boundaries were split, whether model selection touched the test partition, and which review capacity was assumed. Precision-recall analysis is particularly informative for highly imbalanced tasks because it focuses on performance for the rare positive class [Saito and Rehmsmeier, 2015]. Agent assistance raises the provenance burden: fluent explanations from large language models (LLMs) or coding agents can drift from the artifact record unless release decisions are tied to machine-checkable evidence.

Relaytic began as a general local-first inference-engineering lab. Relaytic-AML is the financial-crime edition used here to test whether that architecture can support governed AML experi-

mentation. It is a set of cooperating agents and deterministic harnesses around a local artifact store. The guide helps a user or another agent understand where the run is. The scout checks source posture, schema, leakage risk, and split feasibility. The strategist turns the objective into a task contract. The scientist challenges baselines, ablations, and budget choices. The builder executes bounded runs. Reviewers reconstruct traces. Release governors lint claims, figures, tables, source packages, and public wording against the evidence record.

Relaytic-AML contributes a local evidence and release-governance layer for AML machine-learning experiments. Benchmark rows exercise the architecture under temporal, graph, operating-point, and reporting pressure. The central question is whether a local evaluation lab can keep evidence, agent assistance, privacy boundaries, and admissible interpretation aligned.

The work is organized around four research questions, each scoped to the workflows and deterministic fixtures evaluated here:

- **RQ1:** Can Relaytic-AML produce reproducible evidence cells for AML-style temporal and graph tasks?
- **RQ2:** Do the implemented gates block the tested leakage-prone and unsupported claims?
- **RQ3:** Can the local artifact record support rowless handoff to external agents while preserving provenance?
- **RQ4:** Do the benchmark workflows produce interpretable evidence under explicit split and budget contracts?

The contribution has four linked parts. Evidence cells bind measurements to dataset, split, command, artifact, budget, leakage posture, and operating point. Claim gates govern admissible interpretation separately from the measurement. Rowless handoff exposes state and next actions without exporting raw records. Deterministic failure and ablation fixtures exercise those release boundaries. Local-first software keeps primary data and control on the user’s device while still permitting deliberate collaboration [Kleppmann et al., 2019]. PaySim, Elliptic, and Elliptic2 workflows provide the empirical settings in which these mechanisms are tested.

2 Related Work

The AML benchmark landscape is moving toward larger, more realistic, and more graph-native settings. PaySim remains useful as a synthetic mobile-money fraud simulator for temporal proxy experiments [Lopez-Rojas et al., 2016]. Elliptic introduced a public Bitcoin transaction graph with anonymized node features and temporal labels [Weber et al., 2019]. Elliptic2 and RevTrack/RevClassify shifted attention to suspicious subgraphs and richer blockchain context [Bellei et al., 2024, Song et al., 2024]. Recent preprints such as TransXion, quasi-temporal graph extraction, and BlazingAML, together with published LineMVGNN and continual graph-learning work, treat AML increasingly as a dynamic graph and systems problem [Chen et al., 2026, Poon et al., 2025, Tariq and Hassani, 2026, Ye et al., 2026, Deprez et al., 2025].

Detector papers such as TransXion, BlazingAML, LineMVGNN, and RevClassifyDS push the model frontier. Relaytic-AML sits one layer around that work: it asks how experiments should be governed when data is local or licensed, the task may be temporal or graph-based, analyst review capacity matters, and an agent-assisted workflow must still produce evidence that a skeptical reviewer can audit. The focus on governed local experimentation places Relaytic-AML near dataset documentation, model reporting, reproducibility practice, experiment tracking, and

governance work [Geburu et al., 2021, Mitchell et al., 2019, Pineau et al., 2021, Zaharia et al., 2018].

Experiment-tracking systems preserve runs and artifacts [Zaharia et al., 2018]. Model cards describe trained models, datasheets describe data, reproducibility checklists improve reporting, and agent benchmarks evaluate agent behavior. Relaytic-AML instead connects an executable local measurement to the interpretation that may be released from it. This responsibility matters when evidence comes from licensed files, proxy datasets, temporal graph tasks, or rowless handoff packets rather than from one open leaderboard run.

A closely related newer line uses LLMs and agents for AML triage, graph-context reasoning, suspicious activity report (SAR) narrative support, compliance serving stacks, and runtime agent governance [Pirmorad, 2025, Naik et al., 2025, 2026, Gaurav et al., 2025, Kaptein et al., 2026]. Those systems make agent assistance more capable, but they also make evidence boundaries more important. Relaytic-AML is not a SAR drafting system, not an LLM detector, and not a general-purpose agent-governance product. Its role is narrower: keep local AML evidence, rowless handoff, and public claims aligned.

System distinction. Relaytic-AML places an executable provenance record and a separate interpretation gate between a detector run and every outward-facing table, handoff packet, or claim. Table 1 distinguishes this responsibility from documentation, experiment tracking, detector, and general agent-governance systems.

The adjacent systems remain complementary. Relaytic-AML makes a result reader-facing only after source posture, split, leakage policy, budget, artifact field, handoff posture, and claim boundary are present.

Recent agent-evaluation work shows that language-model systems can produce persuasive research artifacts while still failing on reproduction, validation, or expert-level judgment [Chen et al., 2025, Starace et al., 2025, Wijk et al., 2025]. Skill- and tool-using agents make that opportunity larger and the governance problem sharper [Yang et al., 2026]. Relaytic-AML responds by making model scores, public claims, handoff packets, and paper assets downstream of local artifacts rather than downstream of a conversation transcript.

3 System Overview

Relaytic-AML is built around one authority rule: truth-bearing records live in the local workspace, not in the conversation. Raw data, licensed benchmark files, run summaries, traces, evidence cells, model outputs, tables, figures, and release reports live on disk. Agents may explain, propose, and repair, but their proposals only become evidence when they are materialized as artifacts another human or agent can inspect.

The end-to-end control path is shown in Figure 1. Local inputs enter role-scoped execution, each stage writes typed run artifacts, evidence cells bind reported values to provenance, and release gates determine which interpretations can leave the workspace.

Table 1: Adjacent systems comparison.

Family	Primary object	Relaytic-AML position	Boundary
Model cards and model reporting [Mitchell et al., 2019]	trained model and intended-use report	adds command-level metric provenance, release gating, and stronger-claim blocking around AML experiments	does not replace model reporting
Datasheets and dataset documentation [Gebru et al., 2021]	dataset creation, composition, collection, and recommended use	connects dataset posture to split contracts, leakage controls, benchmark rows, and admissible claims	does not replace source-dataset governance
ML reproducibility checklists [Pineau et al., 2021]	reporting checklist and reproducibility discipline	turns checklist-like obligations into executable artifact-generation and release gates	does not claim full independent reproduction for licensed data
MLOps experiment tracking [Zaharia et al., 2018]	runs, metrics, parameters, artifacts, lineage, and model versions	focuses on local AML evidence, privacy posture, rowless handoff, and public scientific claim admissibility	is not a hosted tracker or production model registry
Agent benchmarks and research-agent evaluations [Chen et al., 2025, Starace et al., 2025]	agent performance on research, coding, or skill-use tasks	uses agents inside a governed local evaluation lab and then tests whether their outputs stay artifact-attached	does not benchmark a general-purpose agent
AML detector and benchmark papers [Weber et al., 2019, Bellei et al., 2024]	detector architecture, benchmark result, graph construction, or financial-crime dataset	provides the local evidence and claim-governance substrate that such detector studies can run through	governs detector evidence rather than introducing a graph-neural model
AML LLM graph reasoning and triage systems [Pirmorad, 2025, Naik et al., 2026]	LLM reasoning, triage, serving, and evidence-rich prompts for AML workflows	keeps LLM or external-agent help downstream of rowless local evidence, artifact provenance, and claim gates	does not claim an LLM detector or AML LLM-serving stack
Agentic SAR and compliance narrative assistants [Naik et al., 2025]	human-in-the-loop SAR or compliance narrative drafting	governs the local experimental evidence and admissible claims that such narrative workflows should cite	does not generate or validate regulatory SAR submissions
Agent governance and runtime trust layers [Gaurav et al., 2025, Kaptein et al., 2026]	runtime policy, enforcement, logging, and trust	specializes governance to AML result provenance, rowless handoff, benchmark context, and public claims	not a general agent-governance platform

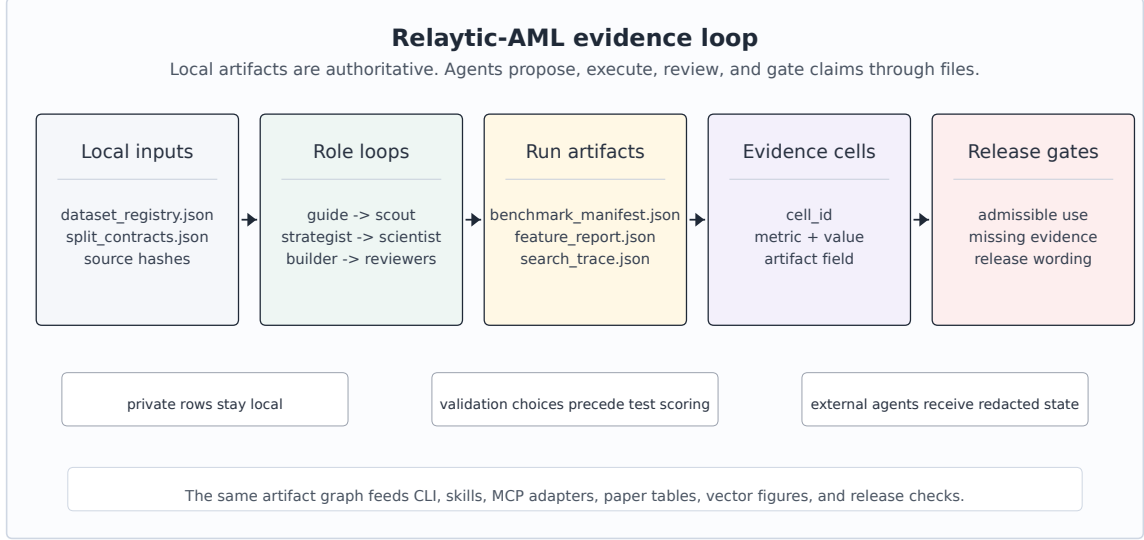


Figure 1: Relaytic-AML local-first architecture: local data and artifacts flow through role-scoped agents into evidence cells, interpretation gates, and paper/release/handoff surfaces.

Figure 1 follows the same state across the system. Dataset registries and split contracts enter the role-scoped runtime. Candidate runs write benchmark manifests, search traces, feature reports, and evidence cells. Release gates read those cells together with audit results and emit only the interpretations supported by the recorded evidence. The same contract feeds the command-line interface, project skills, OpenClaw-style handoff, Claude and Codex skill files, and Model Context Protocol (MCP) adapters.

3.1 Specialist Roles and State

An agent in Relaytic-AML is a capability-scoped role with declared inputs, allowed tools, output artifacts, and an execution budget. The term does not imply that every role is a separate language-model process. Deterministic specialists implement ingestion, profiling, split checks, training, metric computation, redaction, and release validation. Optional language-model backends are reserved for semantic interpretation and guidance, and their output remains advisory until a deterministic artifact contract accepts it.

The roles form a sequence of technical responsibilities. The intake translator converts the request into a task brief. The scout examines source posture, schema, target availability, leakage risks, and split feasibility. The scientist challenges assumptions, baseline strength, ablations, and metric choice. The strategist writes the task, search, and operating-point contracts. The builder executes the approved model route. The challenger and benchmark referee compare alternatives under the same contract. Completion and lifecycle governors decide whether evidence is sufficient, whether another bounded attempt is justified, and whether a candidate can be promoted. Trace adjudicators and release governors reconstruct the path from artifacts and restrict public wording. The guide exposes the same state to a user or an external agent without becoming a second source of truth.

3.2 Harness Execution and Control Loops

The shared harness resolves runtime policy before a specialist runs. A capability profile declares artifact read and write scope, raw-row access, semantic access, and external-adaptor access. Stage start records the active specialist, input artifacts, source surface, and data-access decisions in an append-only event stream. Tool calls are dispatched through a registry that validates names and argument schemas. Stage completion records output artifacts, emits trace events, runs read-only hooks by default, and writes a checkpoint that can be used for recovery or replay.

Language-model-backed turns use a strict action protocol. Each response must be either a structured `tool_call` with validated arguments or a terminal `respond` action. The loop stops when it reaches a response, a policy block, or a user-confirmation boundary. Turn limits, invalid-action limits, repeated-result detection, and consecutive tool-error limits prevent an agent from drifting into an open-ended conversation or repeatedly invoking a failing tool.

Three control loops operate at different levels. The tool-use loop governs one specialist turn. The evaluation loop moves from baseline construction through bounded challenger branches, validation-only selection, audit, and a completion decision. A current follow-up round may request recalibration, retraining, or an alternate challenger, but branch count and round count remain policy-bounded and the incumbent is retained when promotion criteria are not met. The release loop converts metric artifacts into evidence cells, checks leakage and benchmark role, and either emits admissible wording or records the missing evidence required for a stronger claim. These loops communicate through files and events rather than hidden conversational state.

The external-agent path is rowless by default. Relaytic can export current state, next-action options, artifact shortlists, and safe commands for another model without sending raw transaction rows, secrets, or private local paths. A local LLM can optionally help phrase guidance, but the deterministic guide and evidence artifacts remain the source of truth. The separation between advisory help and local evidence matters for private AML work: outside intelligence may help navigate the run, but data residency and claim provenance stay local unless an operator deliberately changes policy.

4 Evidence Cell and Claim-Gate Design

An evidence cell is the unit that makes a paper number auditable. Rather than storing a bare metric value, it records the dataset, split, command, artifact field, model or feature budget, leakage posture, and operating point. Interpretation is deliberately stored in a separate gate output: the cell says what happened, and the gate says how that fact may be used.

The factual fields and their separation from interpretation can be seen in Figure 2.

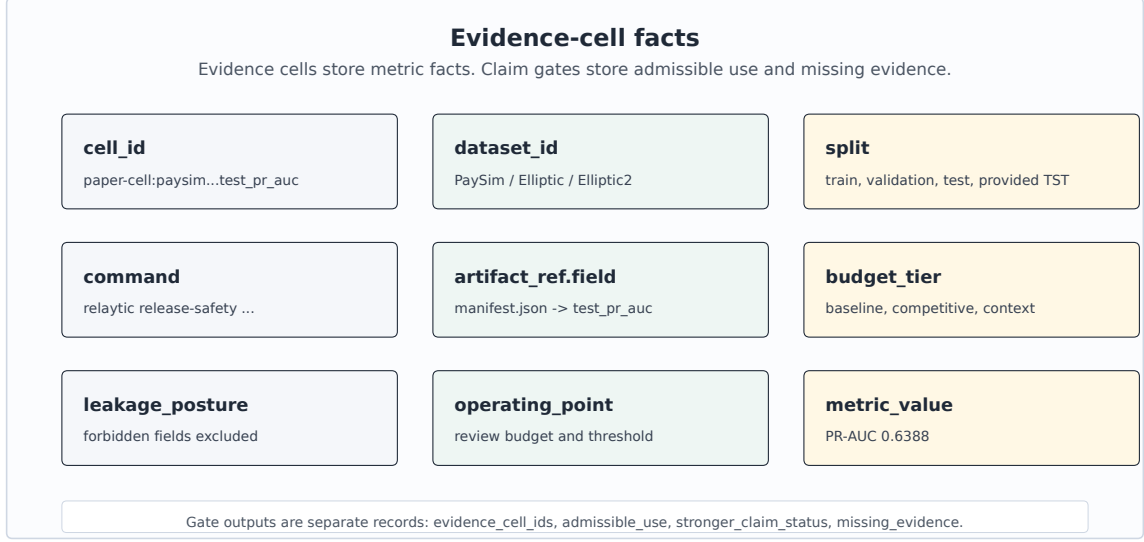


Figure 2: Evidence-cell schema: every reported number carries dataset, split, command, artifact, budget, leakage posture, operating point, metric, and value. Interpretation is stored separately.

Table 2 presents representative evidence cells with compact publication aliases. The underlying evidence records retain the full machine identifiers. Keeping the factual metric record separate from the claim boundary is the central design choice.

Table 2: Representative evidence cells and gate-derived publication roles.

ID	Dataset	Metric	Value	Split	Artifact	Gate-derived use
PS-PR	PaySim	test PR-AUC	0.6388	temporal test	PaySim run, manifest	bounded PaySim temporal-proxy demonstration
PS-P@B	PaySim	precision at review budget	0.7033	temporal test	PaySim run, manifest	bounded PaySim temporal-proxy demonstration
EL-PR	Elliptic	test PR-AUC	0.6688	graph-time test	graph run, feature table	bounded Elliptic temporal graph-feature demonstration
EL-P@B	Elliptic	precision at review budget	1.0000	graph-time test	graph run, feature table	bounded Elliptic temporal graph-feature demonstration
E2-PRm	Elliptic2	provided RevTrack TST PR-AUC mean	0.9432	provided revtrack tst	E2 run, scorecard	Elliptic2 modern benchmark context
E2-ref	Elliptic2	published reference PR-AUC	0.9740	reported ref.	E2 run, scorecard	Elliptic2 modern benchmark context

A representative record is compact enough to audit directly. The public table uses the alias

PS-PR, while the underlying artifact keeps the longer machine identifier. The example shows the factual record that the claim gate later consumes. Stronger interpretations are deliberately kept outside the cell.

Factual evidence cell

```
{
  "cell_id": "PS-PR",
  "dataset_id": "paysim_temporal_transaction_fraud",
  "split": "temporal test",
  "command": "paysim-competitive --budget-tier competitive",
  "artifact_ref": "paper evidence audit: test_pr_auc",
  "metric": "test_pr_auc",
  "value": 0.638773,
  "budget_tier": "competitive",
  "leakage_posture": "balance fields and raw identifiers excluded",
  "calibration_status": "Platt sigmoid calibration",
  "test_exposure_contract": {
    "test_partition_fixed": true,
    "test_partition_previously_exposed": true,
    "competitive_selection_used_test": false,
    "competitive_finalists_tested_after_freeze": 1
  }
}
```

Separate claim-gate record

```
{
  "gate_id": "paysim_p6a_competitive_selected.publication_gate",
  "evidence_cell_ids": [
    "PS-PR"
  ],
  "admissible_use": "bounded PaySim temporal-proxy demonstration",
  "stronger_claim_status": "blocked",
  "gate_reasons": [
    "the fixed test partition had prior P4 reference and P6 baseline exposure",
    "competitive selection did not use test evidence and one finalist was evaluated after  
↪ protocol freeze"
  ],
  "missing_evidence": [
    "genuinely unseen chronological or external holdout",
    "incumbent queue comparison under the same operating contract"
  ]
}
```


Algorithm 1 defines the deterministic construction path from a split contract and bounded run to an auditable evidence cell.

Algorithm 1 Evidence-cell creation

- 1: **Input:** dataset registry D, split contract S, candidate budget B, run artifacts A
 - 2: **Output:** evidence cell c with factual provenance
 - 3: Freeze source posture, license posture, task target, and split contract.
 - 4: Derive only features allowed by S and record excluded leakage fields.
 - 5: Run baseline candidates under the declared baseline budget.
 - 6: Run stronger candidates only within B and select on validation evidence.
 - 7: After validation-only competitive selection and protocol freeze, evaluate the selected competitive finalist on the fixed test partition.
 - 8: Record whether that partition had earlier reference or baseline exposure.
 - 9: Write $c = \text{dataset, split, command, artifact field, metric, value, budget, leakage posture, operating point, and exposure status.}$
 - 10: Hand c to the claim gate before it appears in tables, figures, or release text.
-

The claim gate is the second half of the design. If the evidence cell is incomplete, a split is leakage-prone, a metric is only a proxy, or a stronger interpretation needs a different dataset or study, the gate preserves the evidence and routes the stronger use to an evidence-needs record. The gate is implemented as a release mechanism, so it changes what the paper artifact pipeline and public release surfaces are allowed to say. Algorithm 2 specifies that validation path.

Algorithm 2 Claim-gate validation

- 1: **Input:** public claim q, evidence cells C, gates G, limitations L
 - 2: **Output:** admissible wording and evidence-needs record
 - 3: Resolve every evidence cell named by q and require dataset, split, command, artifact, budget, and leakage fields.
 - 4: Compare the strength of q with source posture, split validity, metric scope, and benchmark role.
 - 5: If q is exactly supported, emit the admissible wording and the evidence-cell identifiers.
 - 6: If q is stronger than C and G permit, record the stronger-claim status and gate reason.
 - 7: Attach the missing evidence needed to make q testable in future work.
 - 8: Route current evidence to its admissible paper use and keep stronger uses out of headline wording.
-

The resulting routes from current evidence to admissible and stronger future uses are shown in Figure 3.

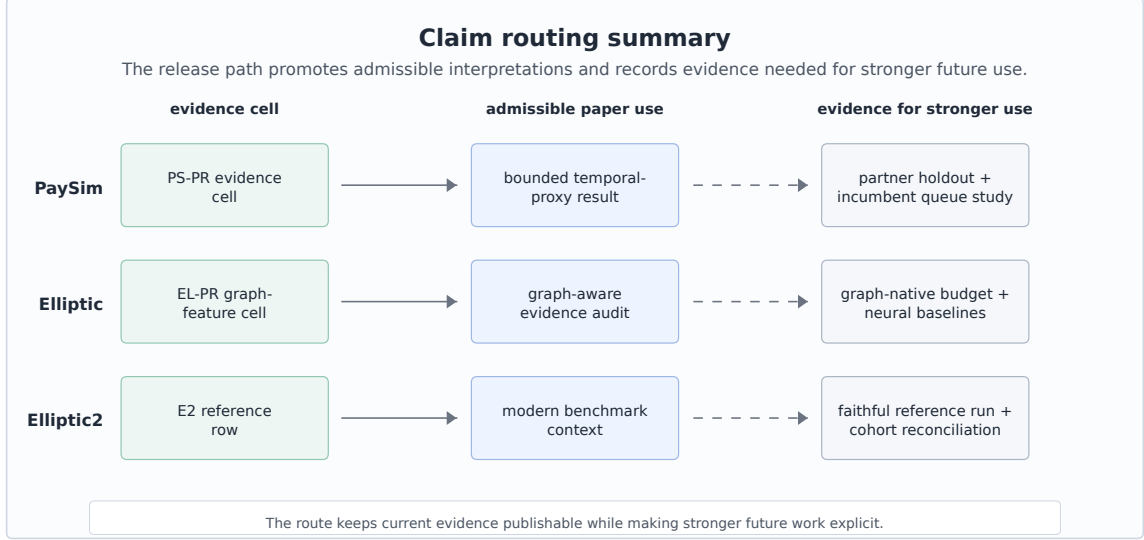


Figure 3: Claim routing summary: current cells map to admissible paper uses and to evidence needed for stronger future interpretations.

Figure 3 makes the routing behavior concrete. A PaySim row becomes a temporal-proxy demonstration, an Elliptic row becomes graph-feature evidence with temporal provenance, and an Elliptic2 row becomes external benchmark context. The same records specify what evidence would be needed before stronger future uses could be made.

5 Experimental Protocol

The experiments test Relaytic-AML as an evaluation lab. The datasets are separated by evidence role. PaySim is the main empirical demonstration because it is fully local and supports a controlled temporal proxy workflow. Elliptic tests temporal graph provenance and feature-view discipline. Elliptic2 tests whether the system can keep a modern external reference row visible without overstating its role.

Dataset scale and exact split boundaries can be seen in Table 3. The corresponding feature, leakage, and metric policies are shown in Table 4. PR-AUC is the primary ranking score because the positive class is rare and analyst capacity is constrained. PaySim uses whole chronological simulator-step boundaries with no gap or embargo. Elliptic uses non-overlapping graph time-step windows. Elliptic2 uses the TRN, VAL, and TST labels supplied by the pinned RevTrack preprocessing artifact. It is reported separately as reference context.

Table 3: Dataset scale and exact split contracts.

Dataset	Unit / positive	Train	Validation	Test
PaySim	transactions / fraud events	1-445: 6,010,937 / 5,007	446-594: 228,103 / 1,552	595-743: 123,580 / 1,654
Elliptic	known-label nodes / illicit nodes	1-29: 26,381 / 2,871	30-39: 8,999 / 1,038	40-49: 11,184 / 636
Elliptic2 context	RevTrack rows / positives	TRN: 88,738 / 2,054	VAL: 11,059 / 252	TST: 11,105 / 272

Elliptic has 203,769 total nodes. Unknown-label nodes are excluded from fitting and metrics. Elliptic2 distinguishes the audited core (121,810 subgraphs) from the RevTrack-evaluable cohort (110,902 rows).

Table 4: Feature, leakage, and metric policy.

Dataset	Split policy	Allowed information	Excluded information	Primary reporting
PaySim	whole chronological steps, no gap or embargo	row-local amount/type/time, prior-step destination history	mixed balance quartet, raw account IDs, simulator flag	PR-AUC, precision/recall at validation threshold
Elliptic	disjoint time windows, metrics on known labels	source features, same-snapshot structure, combined view	future snapshots, unknown labels as targets	PR-AUC, precision/recall at validation threshold
Elliptic2 context	provided RevTrack TRN/VAL/TST labels	pinned pooled subgraph summaries	full-core equivalence not established	repeated PR-AUC, contextual comparison only

Modeling effort is budgeted rather than open-ended. PaySim uses probes on a seeded train-only sample followed by five full-training finalists. Competitive selection, Platt sigmoid calibration, and operating-threshold choice use validation evidence. After protocol freeze, one competitive finalist is evaluated on the fixed test partition [Geurts et al., 2006, Chen and Guestrin, 2016, Platt, 1999]. The same test partition had already produced the P4 reference and P6 baseline rows, so it is fixed but not an untouched holdout. Elliptic compares source-provided anonymized features, Relaytic-derived same-step structural features, and their combination. The selected LightGBM configuration uses seed 42 [Ke et al., 2017]. Elliptic2 uses pooled subgraph summaries with LightGBM seeds 11, 42, and 73 as a context workflow. The model-family and search-budget inventory is kept in the appendix.

For PaySim and Elliptic, finalist or feature-view selection used the full declared validation partition. After that selection, the earlier chronological validation subwindow fitted Platt sigmoid calibration. The later subwindow compared calibrated and identity scores by log loss and selected the review threshold. The two nested subwindows are disjoint, but model selection overlaps both because it used the full validation partition. Their exact boundaries and class counts are reported in the appendix validation-subsplit table.

PaySim contains a mixed balance quartet: `oldbalanceOrig` and `oldbalanceDest` describe pre-transaction balances, whereas `newbalanceOrig` and `newbalanceDest` describe post-transaction balances. Relaytic excludes all four conservatively because their availability and simulator consistency do not match the intended pre-decision contract. Raw account identifiers and simulator flags are also excluded as model inputs. Destination history is computed over the full chronological stream as cumulative activity from strictly earlier steps. It carries across train, validation, and test boundaries, but same-step events do not see one another and future steps cannot contribute. The destination identifier is used only as a grouping key, never as a model feature. Amount thresholds are fitted on training data. The isolated contribution of

this history family has not been tested and is not claimed. For Elliptic, supervised fitting and metrics use only known labels. Unknown-label nodes may contribute observable features and same-step topology, but never targets or metric rows. The source view retains the dataset’s 94 anonymized local features and 72 supplied one-hop neighbor aggregates as one distinct feature family [Weber et al., 2019]. Elliptic contains no edges between time steps, so those supplied neighborhood aggregates are confined to the dataset snapshot. Their construction is inherited from the source and is not attributed to Relaytic. A second family contains Relaytic-derived structural statistics computed only from edges whose endpoints occur in the same time step. The combined view concatenates the two families. No later snapshot contributes to an earlier prediction.

6 Results

The staged PaySim model-selection path and the evidence visible at each stage can be seen in Table 5.

Table 5: PaySim modeling path.

Stage	Model/contract	Selection evidence	Final test evidence	Role
P4 reference	SGD logistic baseline	source-safe starting point	0.2159	reference row
P6 baseline	Extra Trees baseline	leakage-safe feature set	0.3313	baseline row
Probe screen	best small-sample probe: XGBoost	26 allowed features, probe validation PR-AUC 0.5944	no test evaluation	candidate screening
Full finalist selection	Extra Trees finalist	full-training validation PR-AUC 0.5687, selected without test evidence	not evaluated during selection	model selection
Competitive test	Extra Trees with Platt calibration	one finalist after protocol freeze, P4/P6 exposure disclosed	0.6388	bounded demonstration

PaySim is the most complete local modeling path in the current evidence pack. The P4 reference row had test PR-AUC 0.2159, and the P6 leakage-safe baseline reached 0.3313. The competitive search then used validation evidence only. A small-sample XGBoost probe reached validation PR-AUC 0.5944 on a 750,000-row train-only sample. It is not directly rankable against the five finalists, which were refitted on all 6,010,937 training rows. Among those comparable full-training rows, Extra Trees had the highest validation PR-AUC at 0.5687, 0.0282 above the joint XGBoost and Random Forest runner-up rows at the four-decimal precision shown in the table. No tie-break was required to select the winner. One competitive finalist was evaluated after protocol freeze and reached test PR-AUC 0.6388 and ROC-AUC 0.9683. Raw and calibrated test PR-AUC are both 0.6388, so Platt scaling supports probability and threshold handling rather than a ranking-gain claim. The fixed test partition had prior P4 and P6 exposure and is not presented as untouched.

The PaySim operating point was chosen by taking the score at the requested top 0.5% rank on validation and applying that threshold unchanged to test. Test rows with scores equal to the threshold are included. Ties therefore produced a realized test queue of 1,109 of 123,580 transactions (0.8974%), with precision 0.7033 and recall 0.4716. This queue is more concentrated than the 1.3384% test prevalence, but it still misses more than half of the positive test events. The requested fraction and realized queue must therefore be read separately.

Elliptic is a different evidence contract. The validation-selected source-plus-structural LightGBM row has validation PR-AUC 0.9767 and later-window test PR-AUC 0.6688. The gap is consistent with temporal shift, validation-specific selection, or both, but the current artifacts

do not identify a causal decomposition. The same validation-threshold procedure produced a realized test queue of 36 of 11,184 known-label nodes (0.3219%), with precision 1.0000 and recall 0.0566. The difference from the requested 0.5% follows from applying a fixed threshold with ties rather than forcing a test-set rank. This seed-42 point estimate supports temporal graph provenance and operating-point reporting. It does not isolate a graph-detector advance, because source-provided anonymized features strongly influence the selected view. The numerical thresholds, threshold-selection queues, test queues, calibration choices, and tie policy are collected in the appendix operating-point table. In both workflows, the validation-derived threshold is applied unchanged to test, and scores greater than or equal to the threshold are included. No test-set ranking is used to force an exact 0.5% queue.

Elliptic2 is modern benchmark context, not a detector contribution. The audited current core contains 121,810 subgraphs and 2,763 positives, whereas the pinned RevTrack-evaluable table contains 110,902 rows and 2,578 positives. The latter supplies TRN/VAL/TST partitions of 88,738/11,059/11,105 rows. The repeated context estimate on the provided RevTrack TST partition is PR-AUC 0.9432 ± 0.0009 . A separately defined content-hash partition gives mean PR-AUC 0.9297. The provided TST partition had already been inspected during an earlier recovery run, so the repeated value is confirmatory rather than blind or untouched evidence. The published RevClassifyDS full-shot PR-AUC 0.9740 comes from Table 1 of the cited paper and is shown only as an external reference. Cohort equivalence and parity are not established.

Figure 4 separates local ranking evidence, external reference context, and realized review queues. The panels use distinct task contracts and must not be read as a cross-dataset leaderboard.

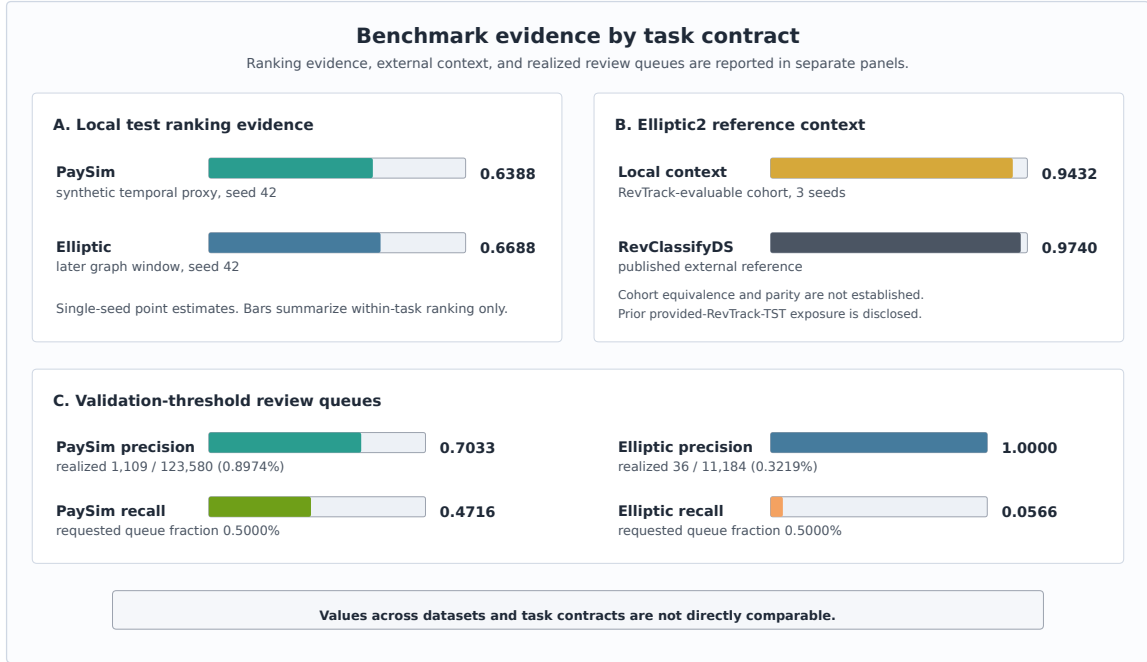


Figure 4: Benchmark evidence by task contract: local ranking estimates, Elliptic2 external-reference context, and validation-threshold review queues are shown in separate panels.

In Figure 4, PR-AUC summarizes ranking within a dataset. Precision and recall describe the test rows selected by a validation-derived threshold, while the realized fractions show how ties and score distributions changed queue size.

7 Deterministic Artifact and Release-Gate Evaluation

The system claim is evaluated through deterministic reader and agent tasks. These checks ask whether a reviewer can navigate from the README to the paper evidence, trace PaySim metric provenance, compare baseline and competitive budgets, keep Elliptic2 as context rather than a contribution, export rowless state for an external agent, recover an interrupted run, and block over-strong public claims. Table 6 reports the synthesis. Detailed failure cases, ablations, the invariant map, the hosted-score example, and handoff rows are preserved in the appendix and generated evidence artifacts.

Table 6: Deterministic artifact and release-gate checks.

Check	Failure condition	Mechanism	Observed result	Scope
Metric provenance	A reported number cannot be traced to source, split, command, or artifact.	Required evidence-cell fields and evidence-cell audit.	14/14 factual fields present, evidence/gate separation audit passed.	Demonstrates traceability on tested paths, not detector optimality.
Budget comparability	Baseline and competitive rows are compared under different contracts.	Dataset, split doctrine, metric, and budget checks.	PaySim PR-AUC changed from 0.3313 to 0.6388 under the same dataset, split, feature, and metric contract, with different declared modeling budgets.	Supports a bounded PaySim comparison, not SOTA.
Leakage and selection firewall	Post-event fields or test evidence influence competitive selection.	Feature policy, validation-only selection, and exposure record.	4 balance fields excluded, competitive selection used no test evidence, one competitive finalist evaluated after protocol freeze, prior P4/P6 exposure recorded.	Fixed partition, not an untouched holdout.
Claim-strength gating	Proxy or context rows become real-bank, parity, or headline claims.	Public wording lint, publishability matrix, and stronger-claim cases.	Six stronger-claim cases tested, hard and headline claims blocked.	Deterministic release gate, not peer review.
Rowless handoff	An external agent receives raw rows, credentials, or private paths.	Context-export redaction and handoff evaluator.	Rowless handoff preserved next action and allowed tools.	Deterministic fixture result, not a privacy certification.
Interrupted recovery	A user or agent cannot recover current state without artifact literacy.	No-lost-user guide and recovery artifact shortlist.	Recovery guide, partial-run state, and artifact shortlist were emitted.	Deterministic recovery check, not a human study.
Hosted-score wrapper	A third-party score file is mistaken for Relaytic detector novelty.	Schema/hash adapter, evidence cell, redaction report, and claim map.	11 exported fields, 16 blocked fields, no raw rows exported	Hosted detector-output governance only.

Across the tested fixtures, Relaytic-AML changes what the release pipeline may promote: a number with missing provenance, a prohibited feature path, a test-selected finalist, an unsafe handoff packet, or an over-strong claim produces a blocked record instead of reader-facing text. These are deterministic infrastructure checks. They are not human usability evidence, privacy certification, or production AML validation.

The hosted external-score fixture shows the intended integration point for stronger third-party detectors. A rowless detector-score artifact enters Relaytic with schema and content hashes, not raw rows. Relaytic emits one governance evidence cell, redacts unsafe handoff fields, and routes the result as hosted detector-output governance evidence. Future detector outputs can

therefore pass through the same local release boundary without being mistaken for a new detector contribution.

8 Limitations and Threats to Validity

PaySim is synthetic. It is useful for controlled temporal fraud experiments, but it is not evidence of bank-scale AML superiority. The simulator has known simplifications, and the current result is a leakage-audited proxy result. The same fixed test partition had already been observed through the P4 reference and P6 baseline before the competitive run. Competitive finalist selection, calibration, and threshold choice remained validation-only, and only one competitive finalist was tested after protocol freeze. Prior exposure nevertheless weakens any untouched-holdout interpretation. A future study should reserve a genuinely unseen chronological or external holdout. The destination-history feature contract is present, but its isolated contribution is not in the current evidence pack.

Public blockchain data is also not the same as bank AML. Elliptic provides a valuable temporal graph task, but unknown labels, anonymized source features, source-supplied neighbor aggregates, and public-chain behavior limit direct operational interpretation. The dataset documentation establishes the one-hop aggregate feature definitions and absence of cross-time-step edges, but Relaytic does not reconstruct the source feature pipeline independently. Elliptic2 is modern and highly relevant, but the current local evidence does not satisfy the reference-parity conditions needed for a performance contribution against RevClassifyDS.

The PaySim and Elliptic detector rows are single-seed point estimates. Prediction-level scores are not part of the committed public evidence pack, so confidence intervals cannot be reconstructed faithfully from aggregate metrics. The deterministic system checks are also not a substitute for a human usability study. They test artifacts, redactions, gate decisions, and recovery surfaces, but do not measure analyst time, production incidents, organizational adoption, or investigation quality. Future work should add repeated runs or a predeclared rowless prediction artifact, private or partner-approved holdouts, same-queue incumbent comparisons, and graph-native families under the same evidence discipline.

The system is intentionally local-first, which creates a tradeoff. Privacy and provenance improve because raw rows stay local, but external reviewers cannot rerun licensed or private data without obtaining it themselves. The paper handles that by publishing commands, hashes where allowed, generated artifacts, and claim boundaries, but a fully independent reproduction of every heavy benchmark still depends on legal access to the source datasets.

9 Reproducibility

The repository is larger than this AML paper. Relaytic is the general local-first inference lab and public package. Relaytic-AML is the focused AML edition used here for the manuscript. A reader should start with the README and this paper. Development-control files record the build history, but they are not required to understand the paper claims. Public citation should use the immutable source commit recorded in the release bundle, or a separately verified public tag, because the main branch can continue to evolve after the paper is posted.

Repository: <https://github.com/ML-Enthusiast-de/Relaytic>. Source commit: e36c7be655a7. Exact release metadata is injected by the clean immutable-revision build.

Table 7 separates what a clean clone can reproduce immediately from what requires local

benchmark access. The README contains the full regeneration script, while the paper keeps the main path short enough to try without reading the generated audit files first.

Table 7: Reproduction modes and dependencies.

Mode	Command fragment	Output class	Requirement
Paper build	paper-release, paper-arxiv-source	Markdown, LaTeX, bibliography, and vector figures	clean clone, TeX required only for PDF compilation
Source validation	paper-final-preflight	citations, logs, fonts, links, metadata, and release gates	compiled PDF and local TeX tools
Deterministic fixtures	paper-invariants	provenance, claim, handoff, and recovery cases	repo-local fixtures, no benchmark data
Artifact verification	paper-release-integrity	metric/split agreement and evidence authority	committed rowless reports, no retraining
PaySim raw-data rerun	paysim-competitive -budget-tier competitive -run-optional -require-full-rerun	competitive model and operating-point artifacts	local PaySim CSV, sha256 prefix 16910f90577b
Elliptic raw-data rerun	graph-baselines -budget-tier competitive -run-optional -require-full-rerun	graph-feature and operating-point artifacts	local Elliptic bundle, sha256 prefix 93e2e7b2405c plus 2 files
Elliptic2 context rerun	elliptic2-competitive -budget-tier competitive -run-suite -require-full-rerun	RevTrack-cohort context artifacts	local Elliptic2/RevTrack files, prior test exposure remains disclosed

The first four rows in Table 7 are available from a clean clone. Artifact verification checks committed evidence without retraining. Deterministic fixtures execute repo-local synthetic cases, and the paper build regenerates publication assets. Raw-data benchmark reruns require local PaySim, Elliptic, or Elliptic2/RevTrack data. A zero process exit from an optional command is not by itself evidence that every benchmark branch ran. Each command emits a machine-readable execution status. Adding `--require-full-rerun` makes the command fail when data, dependencies, or requested branches were skipped.

Minimal public check:

```
python -m pip install -e ".[full]"
python -m relaytic.ui.cli release-safety paper-invariants --format json
python -m relaytic.ui.cli release-safety paper-release --format json
python -m relaytic.ui.cli release-safety paper-narrative-polish --format json
python -m relaytic.ui.cli release-safety paper-novelty-positioning --format json
python -m relaytic.ui.cli release-safety paper-release-integrity --format json
python -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python -m relaytic.ui.cli release-safety paper-final-preflight --format json
```

Raw benchmark data is not committed. PaySim and Elliptic require local downloads and are referenced through registry artifacts, split reports, hashes, and command ledgers. Elliptic2 is used as benchmark context because the stronger reference-parity conditions are not satisfied locally. Clean clones can reproduce paper builds, artifact checks, and repo-local public fixtures. Full benchmark regeneration requires the local datasets named in the README and the fail-on-skip command mode.

10 AI Assistance Disclosure

Large language model tools assisted with drafting, editing, repository inspection, consistency checks, and implementation work around the paper artifacts. They are not authors. Responsibility for the evidence cells, benchmark outputs, source code, figures, tables, limitations, and interpretation remains with the author.

11 Conclusion

Relaytic-AML shows how an agent-assisted AML evaluation lab can be built around local evidence rather than conversational memory. The system records data posture, temporal and graph split validity, leakage controls, model budgets, review-budget operating points, and rowless handoff as factual artifacts. Separate gates govern the interpretations that may be released. The PaySim, Elliptic, and Elliptic2 rows demonstrate that architecture under rare-event, graph-provenance, modern-context, and reporting pressure.

The evidence supports a bounded architectural conclusion: in the workflows and fixtures evaluated here, Relaytic-AML preserved metric provenance, exposed split and operating-point assumptions, produced rowless handoff records, recovered interrupted state, and blocked the tested unsupported claims. Whether those mechanisms improve expert decisions or production outcomes requires human and institutional evaluation. Relaytic-AML is a governance substrate for detector studies rather than a replacement for them.

A Detailed Audit and Reproducibility Records

The appendix keeps concrete audit evidence out of the main reading path while preserving it for reviewers who want to inspect the mechanics. Table 8 records model families and search budgets. The repository stores the corresponding JSON reports with full fields, hashes, and pass criteria.

Table 8: Model families and search budgets.

Track	Families	Features	Search budget	Evidence role
PaySim	tree and boosting candidates, Extra Trees selected	amount, type, time, shifted destination history	14 probes, 5 finalists, seed 42	validation-selected finalist, prior baseline test exposure disclosed
Elliptic	tree/boosting baselines, LightGBM selected	source node features plus same-step graph statistics	16 trials, seed 42	graph-feature evidence row
Elliptic2	LightGBM context row	348 pooled subgraph moments/counts	3 repeated seeds: 11, 42, 73	external reference row

The model-search table records budget shape and evidence role. It is appendix material because it supports auditability without changing the paper’s central architectural claim.

The five comparable PaySim full-training finalists are shown in Table 9. Their validation scores, rather than the small-sample probe scores, determined the competitive winner.

Table 9: Full-training PaySim finalist comparison.

Rank	Finalist	Compact configuration	Validation PR-AUC	Calibration	Test status	Outcome
1	Extra Trees	160 trees, 18 depth, 4 leaf size	0.5687	Platt sigmoid calibration	eligible and evaluated	selected
2=	XGBoost	220 trees, 4 depth, 0.06 learning rate	0.5405	not run	not eligible after validation rank	joint runner-up at displayed precision
2=	Random Forest	120 trees, 18 depth, 5 leaf size	0.5405	not run	not eligible after validation rank	joint runner-up at displayed precision
4	Histogram gradient boosting	160 iterations, 0.08 learning rate, 50 leaf size	0.4674	not run	not eligible after validation rank	lower validation PR-AUC
5	LightGBM	300 trees, 0.03 learning rate, 63 leaves	0.0623	not run	not eligible after validation rank	lower validation PR-AUC

Extra Trees leads the full-training finalist set by 0.0282 PR-AUC over the joint XGBoost and Random Forest runner-up rows at the reported precision. No tie-break was required to select the winner. Nonselected finalists were neither calibrated nor evaluated on test.

Table 10 records the complete operating-point transfer for PaySim and Elliptic.

Table 10: Validation-derived operating-point transfer.

Dataset	Calibration	Threshold	Requested queue	Threshold-selection queue, P/R	Test queue, P/R	Rule
PaySim	Platt sigmoid calibration	0.4045	0.5000%	559/111,601 (0.5009%), 0.5134/0.5053	1,109/123,580 (0.8974%), 0.7033/0.4716	score $\geq$ threshold, equality included
Elliptic	Platt sigmoid calibration	0.9998	0.5000%	21/4,145 (0.5066%), 1.0000/0.0792	36/11,184 (0.3219%), 1.0000/0.0566	score $\geq$ threshold, equality included

The validation threshold is carried unchanged to test and equality is included. Consequently, the realized test queue may differ from the requested validation fraction.

Table 11 reports the validation surfaces used for model selection, calibration, and threshold selection. The full validation partition selected the model, while the nested chronological sub-windows separated calibration fitting from calibration comparison and threshold choice.

Table 11: Validation surfaces for selection, calibration, and thresholding.

Dataset	Purpose	Boundary	Evaluated units	Positives	Overlap and use
PaySim	Model selection	steps 446-594	228,103	1,552	full validation, contains both nested subsets
PaySim	Calibration fit	steps 446-540	116,502	984	earlier subwindow, disjoint from threshold subset
PaySim	Calibration comparison and threshold	steps 541-594	111,601	568	later subwindow, 0.5% threshold selected here
Elliptic	Model and feature-view selection	time steps 30-39	8,999	1,038	full validation, contains both nested subsets
Elliptic	Calibration fit	time steps 30-35	4,854	773	earlier subwindow, disjoint from threshold subset
Elliptic	Calibration comparison and threshold	time steps 36-39	4,145	265	later subwindow, 0.5% threshold selected here

The calibration and threshold-selection subwindows are disjoint. They are not independent of model selection because the complete validation partition was used to rank finalists or feature views.

The injected risks and observed release behavior can be seen in Table 12.

Table 12: Detailed failure-case fixtures.

Failure mode	Injected risk	Gate/check	Evidence	Expected behavior	Observed result
Leakage-column injection	PaySim balance fields are offered as candidate model inputs.	Leakage feature policy	PS-PR feature policy	Post-event balance fields stay out of allowed features.	4 offered, 4 excluded, 0 used, labels not used as features
Test-set selection violation	A model-selection path tries to use test evidence before the finalist is fixed.	Validation-only selection policy	PS-PR search contract	Only validation evidence may select, calibrate, or threshold the finalist. Prior P4/P6 test exposure must remain disclosed.	validation-only probes, no test selection, one competitive finalist evaluated after protocol freeze
Over-strong claim attempt	Draft wording proposes real-bank superiority or RevClassifyDS parity.	Public claim gate	claim-gate report	Unsupported headline and hard-performance claims remain blocked.	6 blocked claims, hard and headline claims blocked
Rowless handoff redaction	An external-agent packet requests raw rows, private paths, or sensitive fields.	Context export redaction	handoff redaction task	The export contains state and next actions, not raw rows or private paths.	raw rows excluded from export, 8 unsafe fields redacted, 6 blocked fields recorded
Interrupted-run recovery	A user or agent resumes a partial run without knowing which artifact to inspect.	No-lost-user guide	guide recovery task	The guide exposes current state, missing evidence, artifact shortlist, and next actions.	partial run recovered, 8 missing-evidence items recorded, 6 recovery actions exposed

The failure-case fixtures exercise whether the release path refuses leakage features, test-set model selection, over-strong claims, unsafe handoff, and lost-run states. They do not add detector benchmark rows.

Table 13 compares the complete governance path with fixtures in which one control is disabled.

Table 13: Governance machinery ablation.

Path	Disabled machinery	Unsafe signal	Artifact integrity	Handoff / recovery	Interpretation
Full governance path	none	0 unsupported claims, leakage inputs, or raw fields	0 missing fields, 3 table groups	6 recovery actions exposed	Claim gate, leakage policy, redaction, provenance, and recovery guide are all active.
No claim gate	public claim gate	6 unsupported claims	3 table groups unchanged	6 recovery actions exposed	The claim gate is what keeps proxy evidence below hard AML, SOTA, RevClassifyDS parity, production, and business-value claims.
No leakage policy	PaySim feature leakage policy	4 leakage inputs	0 missing fields, 1 unsafe table path	6 recovery actions exposed	The leakage policy prevents post-event simulator fields from becoming apparently strong evidence.
No rowless handoff redaction	external-agent redaction	6 raw fields	3 table groups unchanged	6 raw fields, 6 recovery actions exposed	Rowless handoff is the privacy boundary that lets outside agents help without receiving raw data or local paths.
No evidence-cell required fields	evidence-cell required-field gate	10 missing provenance fields	10 missing fields, 1 unsafe table path	6 recovery actions exposed	Required fields connect a reader-facing measurement to its dataset, split, command, artifact, leakage posture, budget, operating-point provenance, and exposure status. Interpretation remains in a separate gate record.
No interrupted-run recovery guide	no-lost-user guide	0 recovery actions	3 table groups unchanged	0 recovery actions exposed	The recovery guide is what keeps state navigation from depending on repo literacy.

These ablations do not rerun detector training. They change the available governance controls and measure the resulting artifact, handoff, recovery, release, and claim states.

The mechanism, stress signal, and boundary associated with each invariant are collected in Table 14.

Table 14: Governance invariants and evidence map.

Invariant	Mechanism	Evidence and stress signal	Boundary
Evidence-cell provenance	evidence-cell audit plus required-field gate	audit: pass, stress ablation: 11 factual provenance fields missing, release blocked	The invariant checks artifact completeness. It does not establish detector optimality.
Claim-strength monotonicity	claim lint, allowed-claims report, publishability matrix, and overclaim failure case	claim lint: pass, stress overclaim stress: 6 unsupported claims blocked	The gate is a deterministic release check, it is not an external peer review.
Leakage and selection firewall	feature policy report, split contract, failure fixtures, and leakage ablation	leakage policy: 4 forbidden fields offered, 0 used, stress leakage stress: 4 leakage fields offered and excluded, 0 used	The current firewall is benchmark-specific, future datasets need their own leakage taxonomy.
Rowless external-agent handoff	handoff evaluator plus redaction failure case	handoff audit: raw rows excluded, 8 unsafe fields redacted, 6 fields blocked, stress redaction stress: 6 unsafe fields blocked, raw rows excluded	The check evaluates deterministic redaction on fixtures. It is not a broad privacy certification.
Interrupted-run recoverability	no-lost-user guide and partial-run recovery fixtures	recovery audit: partial run recovered, 8 missing items, 6 actions exposed, stress recovery stress: partial run recovered, 6 actions exposed	The check is deterministic, it does not measure human time-to-recovery.
Benchmark role separation	publishability matrix and allowed-claims report	role matrix: 5 supporting rows allowed, hard/headline claims blocked, stress claim stress: 6 public claims blocked	Rows with external or proxy roles cannot be treated as unified leaderboard evidence.
Local-first release safety	release go/no-go, claim lint, and public-claim whitelist	wording lint: pass, stress claim boundary: headline and hard performance claims blocked	Licensed benchmark files are not redistributed, reproduction depends on local access.

The invariant map records release-time rules rather than prose preferences. Each invariant pairs a mechanism with an observed stress signal and an explicit boundary.

The hosted external-score path is illustrated by the rowless fixture in Table 15.

Table 15: Hosted external-score case study.

Component	Observed evidence	Evidence	Admissible interpretation
Adapter input	external-score adapter over a rowless fixture, schema hash 0534b78322c7, content hash 8d3c767fd26d	schema/hash report	The score artifact is described by schema and hash posture, not by raw rows.
Evidence emitted	Required metadata fields present, schema-completeness invariant passed	evidence-cell report	This is a governance check, not detector accuracy, ranking performance, or model novelty.
Rowless handoff	11 exported fields, 16 blocked fields, no raw rows exported	handoff-redaction report	A downstream agent can inspect state without receiving rows, identifiers, paths, or secrets.
Claim gate	hosted detector-output governance only, 5 stronger claims blocked	claim-gate report	The public use is hosted detector-output governance only.

Factual hosted-score cell

```
{
  "cell_id": "p19a.external_score.hosted_metadata_completeness",
  "dataset_id": "p19a_hosted_score_fixture",
  "split": "fixture_holdout",
  "command": "relaytic release-safety paper-external-score-proof",
  "artifact_ref": "fixture:p19a_external_score_rowless_v1",
  "metric": "hosted_score_metadata_completeness",
  "invariant_state": "pass",
  "detector_performance_metric": false,
  "leakage_posture": "rowless_no_training_or_label_data_exported",
  "rowless_export_status": "rowless"
}
```

Separate hosted-score gate

```
{
  "gate_id": "p19a.external_score.hosted_output_gate",
  "evidence_cell_ids": [
    "p19a.external_score.hosted_metadata_completeness"
  ],
  "admissible_use": "hosted detector-output governance only",
  "stronger_claim_status": "blocked",
  "gate_reasons": [
    "the fixture measures hosted-score metadata completeness rather than detector
    ↪ performance",
    "raw rows and entity identifiers are excluded from the export"
  ]
}
```

The hosted-score record is metadata governance only. On the tested fixture, a rowless score artifact is wrapped by a factual schema-and-hash record, a redaction report, and a separate interpretation gate. Its completeness result is not detector accuracy or ranking performance.

Table 16 connects stronger future claims to current admissible uses and to the additional evidence each claim would require.

Table 16: Evidence routing examples.

Stronger future use	Current admissible use	Evidence needed
Real-bank deployment study	bounded PaySim temporal-proxy demonstration	Partner or bank-approved holdout, incumbent comparison, analyst-review protocol, and legal release gate.
Elliptic2 reference-method comparison	external RevClassifyDS reference marker plus local context row	Faithful reference execution, cohort reconciliation, resource budget, and repeated parity report.
Graph-native detector release	Elliptic temporal graph-feature evidence path	Graph-native release budget, neural baselines, repeated seeds, and graph-specific ablations.

The blocked-claim rows show how stronger future uses are handled. The gate records current admissible use and the evidence needed before a stronger interpretation could be made.

Concrete external-agent handoff and interrupted-run recovery records are shown in Table 17.

Table 17: Rowless handoff and interrupted-run recovery examples.

Scenario	Input state	Exported fields	Redacted fields	Observed signal
External-agent handoff	partial run with available guide state	state summary, action options, starter questions, tool contract, artifact shortlist	raw transaction rows, credentials, private paths, raw source files	raw rows excluded from export, 8 unsafe fields redacted, 6 blocked fields recorded
Safe next action	external model asked what to do next	six next actions, six starter questions, command options	unredacted local paths and data rows	6 next actions and 6 starter questions exposed
Interrupted-run recovery	operator returns to partial run without artifact literacy	current state, missing evidence count, canonical artifact shortlist, context-export command	raw benchmark data and private machine paths	partial run recovered, 8 missing-evidence items and 6 actions exposed

The handoff and recovery rows give the practical external-agent story. A second model can receive state, commands, artifacts, and starter questions, while raw rows remain redacted and private paths stay withheld.

Appendix reproduction shortcut:

The full Windows and macOS/Linux regeneration script is kept in the README so the appendix remains readable. The essential local paper path is:

Windows PowerShell:

```
py -3.11 -m pip install -e ".[full]"
py -3.11 -m relaytic.ui.cli release-safety paper-invariants --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release --format json
py -3.11 -m relaytic.ui.cli release-safety paper-narrative-polish --format json
py -3.11 -m relaytic.ui.cli release-safety paper-novelty-positioning --format json
py -3.11 -m relaytic.ui.cli release-safety paper-release-integrity --format json
py -3.11 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
py -3.11 -m pytest -m prepush -q
```

After compiling docs/paper/arxiv_src/main.tex and copying the compiled PDF to the final paper location, run `paper-final-preflight`. The README includes the exact compile and verification commands.

macOS/Linux:

```
python3 -m pip install -e ".[full]"
python3 -m relaytic.ui.cli release-safety paper-invariants --format json
python3 -m relaytic.ui.cli release-safety paper-release --format json
python3 -m relaytic.ui.cli release-safety paper-narrative-polish --format json
python3 -m relaytic.ui.cli release-safety paper-novelty-positioning --format json
python3 -m relaytic.ui.cli release-safety paper-release-integrity --format json
python3 -m relaytic.ui.cli release-safety paper-arxiv-source --format json
python3 -m pytest -m prepublish -q
```

References

- Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset, 2024. URL <https://arxiv.org/abs/2404.19109>.
- Hui Chen, Miao Xiong, Yujie Lu, Wei Han, Ailin Deng, Yufei He, Jiaying Wu, Yibo Li, Yue Liu, and Bryan Hooi. MLR-Bench: Evaluating AI Agents on Open-Ended Machine Learning Research, 2025. URL <https://arxiv.org/abs/2505.19955>.
- Keyang Chen, Mingxuan Jiang, Yongsheng Zhao, Zeping Li, Zaiyuan Chen, Weiqi Luo, Zhixin Li, Sen Liu, Yinan Jing, Guangnan Ye, Xihong Wu, and Hongfeng Chai. TransXion: A High-Fidelity Graph Benchmark for Realistic Anti-Money Laundering, 2026. URL <https://arxiv.org/abs/2604.17420>.
- Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL <https://doi.org/10.1145/2939672.2939785>.
- Bruno Deprez, Wei Wei, Wouter Verbeke, Bart Baesens, Kevin Mets, and Tim Verdonck. Advances in Continual Graph Learning for Anti-Money Laundering Systems: A Comprehensive Review. *WIREs Computational Statistics*, 17(3):e70040, 2025. doi: 10.1002/wics.70040. URL <https://doi.org/10.1002/wics.70040>.
- Federal Financial Institutions Examination Council. BSA/AML Examination Manual: Appendix F, Money Laundering and Terrorist Financing Red Flags, 2014. URL <https://bsaaml.ffiec.gov/manual/Appendices/07>. Accessed 2026-07-14.
- Financial Action Task Force. Virtual Assets Red Flag Indicators of Money Laundering and Terrorist Financing, 2020. URL <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Virtual-assets-red-flag-indicators.html>. Accessed 2026-07-14.
- Suyash Gaurav, Jukka Heikkonen, and Jatin Chaudhary. Governance-as-a-Service: A Multi-Agent Framework for AI System Compliance and Policy Enforcement, 2025. URL <https://arxiv.org/abs/2508.18765>.
- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. Datasheets for Datasets. *Communications of the ACM*, 64(12):86–92, 2021. doi: 10.1145/3458723. URL <https://arxiv.org/abs/1803.09010>.

- Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely Randomized Trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1. URL <https://doi.org/10.1007/s10994-006-6226-1>.
- Maurits Kaptein, Vassilis-Javed Khan, and Andriy Podstavnychy. Runtime Governance for AI Agents: Policies on Paths, 2026. URL <https://arxiv.org/abs/2603.16586>.
- Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.
- Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-First Software: You Own Your Data, in Spite of the Cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, pages 154–178, 2019. doi: 10.1145/3359591.3359737. URL <https://doi.org/10.1145/3359591.3359737>.
- Edgar Alonso Lopez-Rojas, Ahmad Elmir, and Stefan Axelsson. PaySim: A Financial Mobile Money Simulator for Fraud Detection. In *Proceedings of the 28th European Modeling and Simulation Symposium*, 2016. URL https://www.msc-les.org/proceedings/emss/2016/E_MSS2016_249.pdf.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model Cards for Model Reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596. URL <https://arxiv.org/abs/1810.03993>.
- Prathamesh Vasudeo Naik, Naresh Kumar Dintakurthi, Zhanghao Hu, Yue Wang, and Robby Qiu. Co-Investigator AI: The Rise of Agentic AI for Smarter, Trustworthy AML Compliance Narratives, 2025. URL <https://arxiv.org/abs/2509.08380>.
- Prathamesh Vasudeo Naik, Naresh Dintakurthi, and Yue Wang. Rethinking LLMOps for Fraud and AML: Building a Compliance-Grade LLM Serving Stack, 2026. URL <https://arxiv.org/abs/2605.11232>.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. Improving Reproducibility in Machine Learning Research. *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL <https://www.jmlr.org/papers/v22/20-303.html>.
- Erfan Pirmorad. Exploring the In-Context Learning Capabilities of LLMs for Money Laundering Detection in Financial Graphs, 2025. URL <https://arxiv.org/abs/2507.14785>.
- John C. Platt. Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.
- Chung-Hoo Poon, James Tin Yau Kwok, Calvin Chow, and Jang-Hyeon Choi. LineMVGNN: Anti-Money Laundering with Line-Graph-Assisted Multi-View Graph Neural Networks. *AI*, 6(4):69, 2025. doi: 10.3390/ai6040069. URL <https://doi.org/10.3390/ai6040069>.

- Takaya Saito and Marc Rehmsmeier. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*, 10(3): e0118432, 2015. doi: 10.1371/journal.pone.0118432. URL <https://doi.org/10.1371/journal.pone.0118432>.
- Kiwhan Song, Mohamed Ali Dhraief, Muhua Xu, Locke Cai, Xuhao Chen, Arvind, and Jie Chen. Identifying Money Laundering Subgraphs on the Blockchain. In *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024. doi: 10.1145/3677052.3698635. URL <https://arxiv.org/abs/2410.08394>.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI’s Ability to Replicate AI Research, 2025. URL <https://arxiv.org/abs/2504.01848>.
- Haseeb Tariq and Marwan Hassani. Extracting Money Laundering Transactions from Quasi-Temporal Graph Representation, 2026. URL <https://arxiv.org/abs/2604.02899>.
- Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics, 2019. URL <https://arxiv.org/abs/1908.02591>.
- Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents against Human Experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832, 2025. URL <https://proceedings.mlr.press/v267/wijk25a.html>.
- Yifan Yang, Ziyang Gong, WeiQuan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive Strategy for Self-Evolving Agent Skills, 2026. URL <https://arxiv.org/abs/2605.23904>.
- Haojie Ye, Arjun Laxman, Yichao Yuan, Krisztian Flautner, and Nishil Talati. BlazingAML: High-Throughput Anti-Money Laundering (AML) via Multi-Stage Graph Mining, 2026. URL <https://arxiv.org/abs/2604.12241>.
- Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. Accelerating the Machine Learning Lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45, 2018. URL https://people.eecs.berkeley.edu/~matei/papers/2018/ieee_mlflow.pdf.